

Neural Style Transfer

CSCI 611 | Assignment 4

Amol Bhalerao | March 2026

1. Introduction

Neural Style Transfer (NST) is an optimization-based image synthesis technique introduced by Gatys, Ecker, and Bethge (CVPR 2016). Given a content image and a style image, NST synthesises a new target image that retains the high-level structure of the content while adopting the visual texture and artistic appearance of the style image.

The algorithm exploits the feature hierarchy of a pre-trained VGG19 convolutional network: deep layers encode semantic content, while Gram matrices of activations at multiple layers encode multi-scale style and texture statistics. Only the target image pixels are updated by gradient descent -- the network weights remain frozen throughout.

Reference: Gatys, Ecker & Bethge. Image Style Transfer Using Convolutional Neural Networks. CVPR 2016.

https://www.cv-foundation.org/openaccess/content_cvpr_2016/papers/Gatys_Image_Style_Transfer_CVPR_2016_paper.pdf

2. Implementation

2.1 Images Used

Role	Description	Source
Content	Labrador Retriever photograph	Wikimedia Commons (CC BY-SA 3.0)
Style	The Starry Night - Van Gogh (1889)	Wikimedia Commons (Public Domain)

2.2 Network: VGG19 (Frozen Features)

Only the .features portion of VGG19 is used (all convolutional and pooling layers). All parameters are frozen (requires_grad = False). The model is moved to GPU if available, otherwise CPU is used.

```
vgg = models.vgg19(weights=VGG19_Weights.DEFAULT).features
for param in vgg.parameters():
    param.requires_grad_(False)
vgg.to(device)
```

2.3 VGG19 Layer Mapping (TODO 1 - Completed)

The paper specifies which VGG19 layers to use for content and style representations. The mapping from PyTorch layer indices to the paper names is shown below.

Paper Name	PyTorch Index	Role
conv1_1	0	Style
conv2_1	5	Style

conv3_1	10	Style
conv4_1	19	Style
conv4_2	21	Content (semantic structure)
conv5_1	28	Style

```

layers = {
    '0': 'conv1_1', # style
    '5': 'conv2_1', # style
    '10': 'conv3_1', # style
    '19': 'conv4_1', # style
    '21': 'conv4_2', # content
    '28': 'conv5_1', # style
}

```

2.4 Gram Matrix (TODO 2 - Completed)

The Gram matrix captures style by measuring pairwise correlations between all feature maps at a given layer. For a tensor of shape (batch, d, h, w):

- Reshape to (d, h x w): flatten spatial dims so each row corresponds to one filter.
- Compute $G = F \times F^T$: the (d x d) matrix captures which feature maps co-activate.
- This is spatially invariant -- encodes WHAT co-occurs, not WHERE.

```

def gram_matrix(tensor):
    batch_size, d, h, w = tensor.size()
    tensor = tensor.view(batch_size * d, h * w) # flatten spatial dims
    gram = torch.mm(tensor, tensor.t()) # (d x d) correlation matrix
    return gram

```

2.5 Loss Functions (TODO 3 - Completed)

Content Loss: MSE between target and content feature maps at conv4_2.

```

content_loss = torch.mean(
    (target_features['conv4_2'] - content_features['conv4_2'])**2
)

```

Style Loss: Weighted MSE between Gram matrices of target and style images at all five style layers, normalized by each layer's size (d x h x w). Total Loss combines both: $\text{total} = \alpha * \text{content_loss} + \beta * \text{style_loss}$.

```

style_loss = 0
for layer in style_weights:
    target_gram = gram_matrix(target_features[layer])
    style_gram = style_grams[layer]
    _, d, h, w = target_features[layer].shape
    layer_style_loss = style_weights[layer] * torch.mean((target_gram - style_gram)**2)
    style_loss += layer_style_loss / (d * h * w)

total_loss = content_weight * content_loss + style_weight * style_loss

```

2.6 Optimization Loop (TODO 4 - Completed)

The target image is initialized as a clone of the content image and updated using Adam optimizer for 2000 steps. VGG19 weights are never modified.

```
target      = content.clone().requires_grad_(True).to(device)
optimizer = optim.Adam([target], lr=0.003)

for ii in range(1, steps + 1):
    target_features = get_features(target, vgg)
    content_loss    = torch.mean(
        (target_features['conv4_2'] - content_features['conv4_2'])**2
    )
    # compute style_loss across 5 layers (see above)
    total_loss = content_weight * content_loss + style_weight * style_loss
    optimizer.zero_grad()
    total_loss.backward()
    optimizer.step()
```

3. Hyperparameter Experiments (Part 2)

A helper function `run_style_transfer()` was implemented to run each experiment cleanly. Four experiments were conducted, varying one hyperparameter at a time while keeping the rest at defaults: `style_weight=1e6`, `lr=0.003`, `steps=2000`, content-image initialisation.

Experiment 1 - Style Weight Ratio (beta)

style_weight	Effect on Output
1e4 (content-heavy)	Closely resembles the original photo; minimal stylisation
1e6 (balanced)	Clear content structure with visible Van Gogh swirls
1e8 (style-heavy)	Abstract painterly output; content structure mostly lost

Finding: $\beta=1e6$ gives the best tradeoff between content fidelity and style strength. This is the single most impactful hyperparameter. See: `exp1_style_weight.png`

Experiment 2 - Style Layer Weight Configuration

Configuration	Weight Pattern	Visual Effect
Early emphasis	High: conv1_1, conv2_1	Coarse brush strokes; strong colour transfer
Uniform	Equal across 5 layers	Balanced mix of fine and coarse texture
Late emphasis	High: conv4_1, conv5_1	Fine-grained texture; subtler colour shift

Finding: Early layers capture colours and edge orientations. Weighting them more heavily produces the most visually striking style transfer. See: `exp2_layer_weights.png`

Experiment 3 - Starting Image Initialisation

Initialisation	Convergence	Visual Result
Content image (default)	Fast	Content preserved; style cleanly overlaid
Random Gaussian noise	Slow (2-3x more steps)	Abstract; style-dominant output

Finding: Content initialisation provides a warm start, halving convergence time and preserving semantic structure. See: `exp3_init.png`

Experiment 4 - Adam Learning Rate

Learning Rate	Convergence Behaviour	Output Quality
0.001 (slow)	Under-converges in 2000 steps	Under-stylised result
0.003 (default)	Smooth and steady decrease	Best quality - sweet spot
0.010 (fast)	Rapid initial convergence	Slight noise from overshooting

Finding: $lr=0.003$ is the optimal rate for 2000 steps. Higher LR overshoots the minimum causing noise; lower LR does not converge enough. See: `exp4_lr.png`

4. Summary of Hyperparameter Experiments

Hyperparameter	Default	Low Effect	High Effect	Best
style_weight (b)	1e6	Preserves content	Loses content	1e6
Layer weights	Gatys et al.	Less coarse texture	Less fine texture	Early layers
Initialisation	Content	--	--	Content clone
Learning rate	0.003	Under-converges	Noise artefacts	0.003

Key takeaways from all experiments:

- The content/style weight ratio (alpha/beta) is by far the most impactful hyperparameter.
- Using Gram matrices from 5 VGG layers (multi-scale) is essential for realistic texture transfer.
- Gram matrices capture texture statistics in a spatially-invariant way -- the core insight of the paper.
- Content-image initialisation halves convergence time versus starting from random noise.
- GPU acceleration is critical: 2000 steps take ~2 min on GPU vs 20-40 min on CPU.

5. Conclusion

The Gatys et al. (2016) neural style transfer algorithm was successfully implemented in PyTorch using a frozen VGG19 backbone. All four TODOs were completed:

- TODO 1: Content and style image loading with correct VGG preprocessing.
- TODO 2: VGG19 layer mapping -- conv1_1 through conv5_1 for style; conv4_2 for content.
- TODO 3: Gram matrix computation capturing pairwise feature-map correlations.
- TODO 4: Content loss, style loss, and total loss with the full Adam optimization loop.

Four systematic hyperparameter experiments confirmed the paper recommendations. The final stylised output (Labrador + Starry Night) clearly shows recognisable dog features rendered in Van Gogh's iconic swirling brush-stroke style, demonstrating that the implementation is both correct and effective.

References

- [1] L. A. Gatys, A. S. Ecker, and M. Bethge. Image Style Transfer Using Convolutional Neural Networks. IEEE CVPR, 2016.
https://www.cv-foundation.org/openaccess/content_cvpr_2016/papers/Gatys_Image_Style_Transfer_CVPR_2016_paper.pdf
- [2] K. Simonyan and A. Zisserman. Very Deep Convolutional Networks for Large-Scale Image Recognition. arXiv:1409.1556, 2014.
- [3] PyTorch Documentation. torchvision.models.vgg19.
<https://pytorch.org/vision/stable/models/generated/torchvision.models.vgg19.html>